qmath.cを理解する

数字の扱い

- uint32_t
  - 符号なしの32bit
  - 0 〜 4,294,967,295 ($2^{32} - 1$)
  - q16.16のraw値, アドレス, ビット演算用
- int64_t
  - 符号付きの64bit
  - -9,223,372,036,854,775,808 〜 9,223,372,036,854,775,807
  - q16_mulなどで, 32bit同士を掛けたときに一時的に数値が巨大化してもオーバーフローが起こらないようにする
- q16.16
  - 整数部:16bit, 少数部:16bit
  - raw値:内部では本来の数値の2^16倍された数値として記録されている
  - 約 -32768.0 〜 $+32767.99998...$
  - 0x80000000　〜 0x7FFFFFFF
  - 実数値1.0
    - $1.0 * 65536 = 65536$
    - 0x00010000
  - 実数値2.5
    - $2.5 * 65536 = 136840$
    - 0x00028000
  - 実数値-1.0
    - $-1.0 * 65536 = -65536$
    - 0xffff0000

関数の扱い

- sx32
  - uint32_tをint64_tに変換
  - 正の最大値(0x7fffffff)以下であれば、そのまま正の値
  - それ以上だと、負の値
- floor_div_pow2
  - 床関数付き2の冪割り算
  - 2のn乗で割り算をしつつ、結果を正しい床に合わせる
  - -5 / 2はどうなる?
    - C言語標準:-5 / 2 = -2 > 誤り(0方向に丸め込んでしまう)
    - 正しくする:-5 / 2 = -3 > 正解

- q16_mul
    - q16.16同士の掛け算
    - sx32で両方を64bitに拡張して掛け算
    - floor_div_pow2で右に16bitシフトし、もとに戻す
- i32_div
    - 分子が0であるときは, エラー値0xFFFFFFFFUを返す
    - 0x80000000U(表現できる最も小さなマイナスの値 2^{-31})を-1で割るとオーバーフローを起こすのでブロック
    - その他の場合は、普通に計算
- i32_rem
    - 分子が0とき, aを返す
    - 2^{-31}を-1で割るときのあまりは, 0
    - その他の場合は、普通に計算
- q16_rcp
    - q16.16の固定少数点における逆数
    - 0を割ろうとするときは, 最大の正の値(0x7fffffffu)を返す
    - Q16.16のスケールを合わせるために、分子に 2^{32}（1LL << 32）を置いて割り算
    - 逆数の計算結果がマイナスでかつ余りがある場合, floor補正
    - 32bitの範囲を超えた場合のケア